

LLM 기반 왕복 번역을 활용한 프로그래밍 언어 간 거리의 정량적 측정

한상곤

2026년 3월

Abstract

본 연구는 왕복 번역(Round-Trip Translation, RTT) 기법과 대형 언어 모델(LLM)을 활용하여 프로그래밍 언어 간의 의미적 거리를 정량적으로 측정하는 방법을 제시한다. 기준이 되는 C++ 코드를 C, Java, Python으로 번역하고 이를 다시 원본 언어로 역번역하는 순환 과정을 통해 의미적 거리를 정량적으로 측정했다. qwen2.5-coder:7b와 gpt-5.4를 대상으로 총 30건의 실험을 진행하였고, 두 모델은 뚜렷한 차이를 드러냈다. Qwen은 원본의 구문적 구조를 최대한 보존하려는 보수적 전략을 취하며 언어별 편차 없이 평균 53.3%의 균형 잡힌 성공률을 기록했다. 반면, GPT-5.4는 cpp→c 변환 과정에서 의미적 등가성을 100% 완벽하게 유지하면서도, 원본의 구조에 얽매이지 않고 대상 언어의 패러다임에 맞게 코드를 전면 재구성하는 경향을 보였다. 본 연구는 정점(Fixed-point) 도달 여부, 반복 횟수, 잔차 유사도(Residual Similarity), 추상 구문 트리(AST) 기반의 구조적 거리, 그리고 MOSS 유사도 등 다차원적인 지표를 통합하여 분석했다. RTT 프레임워크는 향후 소프트웨어 엔지니어링에서 이기종 언어 간의 코드 마이그레이션 난이도를 사전에 예측하고, AI 기반 코드 변환 도구의 신뢰성을 검증하는 지침과 도구가 될 것으로 기대한다.

키워드: 왕복 번역(Round-Trip Translation), 프로그래밍 언어 거리, 대규모 언어 모델(LLM), 코드 변환, AST 편집 거리, 잔차 유사도

1 서론

1.1 연구 배경

프로그래밍 언어 간의 거리를 정량적으로 측정하려는 시도는 프로그래밍 언어론과 소프트웨어 공학의 오랜 주제이다. 전통적으로 언어 간 유사도는 구문(syntax) 계열, 타입 시스템, 패러다임(절차적, 객체지향적, 함수형) 등의 정성적 분류에

의존해 왔다. 그러나 이러한 분류는 실제 코드 변환 과정에서 나타나는 구조적, 의미적 변형의 정도를 정확히 반영하지 못한다.

한편, 대규모 언어 모델(LLM)의 코드 생성 및 번역 능력이 급속히 발전하면서, LLM을 번역기로 활용하여 언어 간 변환을 수행하고 그 과정에서 발생하는 변형을 측정하는 새로운 접근이 가능해졌다. 자연어 처리 분야에서 왕복 번역(round-trip translation)은 기계 번역 품질을 평가하는 고전적 기법이며, 이를 프로그래밍 언어에 적용하면 언어 간 거리를 반영하는 정량적 지표를 도출할 수 있다는 가설이 본 연구의 출발점이다.

1.2 연구 목적

본 연구의 목적은 다음 세 가지로 요약된다.

- 1) RTT 기반 언어 거리 측정 프레임워크 구축: C++를 기준 언어(seed language)로 설정하고, C, Java, Python으로 왕복 번역 순환을 통해 언어 간 거리를 다차원적으로 측정하는 자동화된 실험 파이프라인을 설계, 구현한다.
- 2) 이종 LLM 간 비교 분석: 로컬 오픈소스 모델(qwen2.5-coder:7b)과 상용 클라우드 모델(gpt-5.4)의 번역 행동을 동일 코퍼스 동일 조건에서 비교하여, 모델 특성이 언어 간 거리 측정 결과에 미치는 영향을 분석한다.
- 3) 다차원 지표의 종합적 해석: 수렴 속도(반복 횟수), 잔차 토큰 유사도, AST 편집 거리, MOSS 표절 검사 유사도, 의미 보존 여부(semantic pass/fail) 등 이질적 지표들의 상호 관계를 분석하여, 각 지표가 드러내는 언어 간 거리의 서로 다른 측면을 조명한다.

2 실험 설계

2.1 실험 코퍼스

실험 대상 코퍼스는 백준 온라인 저지(Baekjoon Online Judge)에서 선별한 5개 알고리즘 문제로 구성된다. 각 문제는 서로 다른 알고리즘 패턴을 포함하도록 선택하였다.

각 문제에는 10쌍의 입출력 샘플 픽스처(fixture)가 제공되어, 번역된 코드의 의미적 정확성(semantic correctness)을 자동으로 검증할 수 있도록 하였다.

2.2 기준 언어 및 대상 언어

- 기준 언어(seed language): C++
- 대상 언어(target languages): C, Java, Python

Table 1. 실험 코퍼스: 대상 문제 목록

문제 ID	문제명	알고리즘 특성
1436	영화감독 숨	문자열 탐색, 반복문
2110	공유기 설치	이분 탐색
2217	로프	정렬, 그리디
2579	계단 오르기	동적 프로그래밍
5567	결혼식	정렬, 구간 처리

- 언어 쌍(ordered pairs): `cpp→c`, `cpp→java`, `cpp→python`

C++를 기준 언어로 선택한 이유는 (1) 세 대상 언어 모두와 구문적으로 어느 정도 공통 요소를 공유하면서도, (2) 각 언어로의 변환 시 서로 다른 수준의 구조적 변형이 필요하기 때문이다. 특히 C는 C++의 부분 집합에 가까운 구문 유사성을, Java는 객체지향 래핑의 필요성을, Python은 근본적으로 다른 구문 규칙(들여쓰기 기반, 동적 타이핑)을 각각 대표한다.

2.3 실험 모델

두 가지 LLM을 사용하여 동일 실험을 독립적으로 수행하였다.

Table 2. 실험에 사용된 LLM 모델 사양

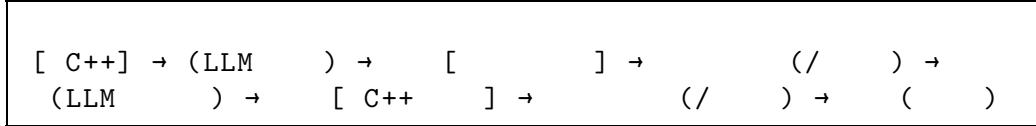
항목	Ollama	OpenAI
모델명	qwen2.5-coder:7b	gpt-5.4
유형	로컬 오픈소스	상용 클라우드 API
파라미터 규모	7B	비공개(대규모)
온도(temperature)	0	0
실행 환경	로컬 머신	클라우드 API
실행 ID	smoke-ollama-001	smoke-openai-001

두 모델 모두 temperature를 0으로 고정하여 결정론적(deterministic) 출력을 추구하였다. 다만, temperature = 0이더라도 완전한 재현성은 보장되지 않는다는 점에 유의해야 한다.

2.4 왕복 번역 파이프라인

실험 파이프라인의 한 주기(cycle)는 다음과 같은 단계로 구성된다.

Listing 1. RTT 파이프라인 한 주기



수렴 판정(convergence detection)은 다음 규칙에 따른다.

- 고정점(fixed_point): 마지막 두 왕복 C++ 코드의 정규화된 해시가 동일
- 진동(oscillation): 마지막 네 해시가 A, B, A, B 패턴을 보이며 $A \neq B$
- 실패 종료(terminated_on_failure): 컴파일 오류, 런타임 오류, 오답, 타임아웃 등으로 중단
- 최대 반복 한도: 20회

2.5 측정 지표

각 실험 단위(문제--언어 쌍)에 대해 다음 지표를 수집하였다.

- 1) 변경 횟수 거리(change-count distance, CCD): 완료된 왕복 주기 수. 값이 클수록 수렴에 오래 걸림을 의미한다.
- 2) 잔차 유사도(residual similarity): 원본 C++와 최종 왕복 C++의 정규화된 토큰 멀티셋 간 Sørensen--Dice 계수(식 1). 값이 1에 가까울수록 원본 보존율이 높다.

$$DSC(X, Y) = \frac{2|X \cap Y|}{|X| + |Y|} \quad (1)$$

- 3) AST 편집 거리(AST distance): tree-sitter로 파싱한 뒤 언어 독립적 IR(중간 표현)로 정규화하고, APTED(All Path Tree Edit Distance) 알고리즘으로 계산한 트리 편집 거리.
- 4) MOSS 유사도: Stanford MOSS(Measure of Software Similarity) 시스템을 활용한 코드 유사도 측정. seed(원본 C++ 대비)와 roundtrip(왕복 C++ 대비) 두 방향을 기록한다.
- 5) 의미 보존 여부(semantic pass/fail): 대상 언어 코드와 왕복 C++ 코드가 모든 샘플 입출력을 통과하는지 여부.
- 6) 복잡도 델타(complexity delta): lizard 도구로 측정한 LOC, token_count, cyclomatic_complexity, function_count, max_nesting_depth의 원본 대비 변화량.